AI × ROBOTICS / TASK & OBJECTIVE

VLA推論とアーム制御の非同期化

SmolVLAとπ0.5による、ボウル移載タスクの比較

ロボットが画像と言語指示から行動列を生成し、黒いボウルをつかんで皿へ置く。LIBERO向けに調整済みの公開モデルを用い、本制作では追加学習せず、センサ・ROS 2・方策・シミュレータ内の制御を接続した。

① 開始時の配置



② ボウルを皿へ移載した状態



π0.5・非同期・配置1の実行記録。各画像の左は外部カメラ、右は手首カメラ。成否はLIBEROの公式判定を使用。

目的と検証した仮説

推論中も制御周期を維持するシステムを構築し、モデルの処理時間と操作成功の関係を調べた。計画時は、小型SmolVLAの低遅延が非同期操作に有利と予想した。そこで、同じ課題・初期配置・行動実行数でπ0.5と比較した。

同期実行の成功

5/5 : 5/5

SmolVLA : π0.5
両者とも全配置で成功

非同期実行の成功

3/5 : 5/5

SmolVLA : π0.5
実経過14秒以内の評価

非同期中の方策生成・出力処理

508 / 332 ms

SmolVLA / π0.5
入力前処理を除く試行平均

制作の中心：モデルを実時間システムへ接続する設計

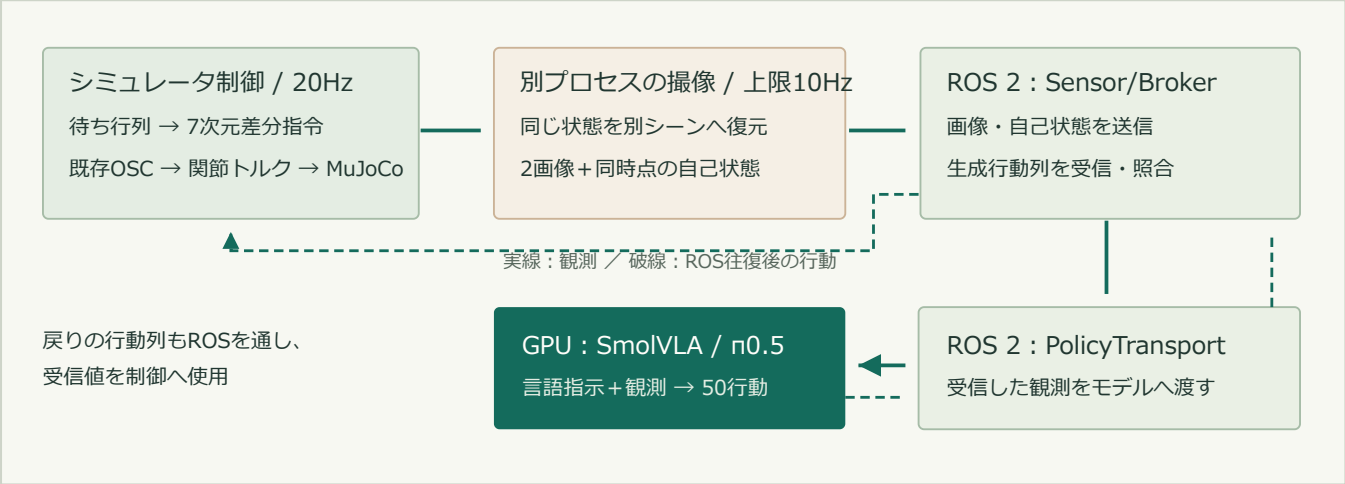
既存のPanda・接触物理・学習済み方策を再利用し、撮像、推論、制御を別プロセスへ分離した。取得時刻の管理、行動の待ち行列、ROS受信値の照合を実装し、処理時間と操作結果を対応づけて検証した。

環境	LIBERO Spatial task0 / MuJoCo	**機体**	Pandaアーム＋グリッパ
計算機	RTX 5080・VRAM 16GB / WSL2	**評価範囲**	既知の1課題・固定5配置

SYSTEM / SENSOR → POLICY → ACTUATOR

センサ・ROS・制御の接続と計算資源の管理

外部・手首画像、ロボット自身の状態、言語指示をVLAへ入力する。モデルは相対位置3・姿勢3・グリッパ1の行動列を出力し、既存の操作空間制御器（OSC、位置・姿勢のPD制御）が関節トルクへ変換する。



行動列は複数時点の指令をまとめたもの（action chunk）。画像・自己状態と行動列は、実際にROSの別ノード間で送受信し、受信した値を使用する。物体の正解位置は評価器だけが使用する。

撮像を制御周期から分離

2カメラの同期描画は平均85.64msで、20Hzの制御予算50msを超えた。画像なしのシミュレータ更新は8.98msだったため、状態を複製するカメラプロセスへ描画を分離した。

取得時刻と待ち行列の管理

画像と自己状態を同じsnapshotから作り、取得時刻を保持する。カメラの待ち行列は長さ1、推論要求は同時に1件とし、古い観測の蓄積を抑えた。

ハードウェア・プロセス管理の実装

GPU	モデルを1種類ずつ実行。RTC追加系列と推論内訳計測では、別実験とGPU利用を調整し、全体使用量も監視した。
CPU・メモリ	推論側4スレッド、撮像側BLAS 1スレッド。ROSコンテナはCPU 2コア相当・メモリ2GiBへ制限した。
主比較のモデル最大割当	SmolVLA 1.47GiB / π0.5 8.87GiB。PyTorchのモデルプロセス内の最大値で、描画・デスクトップ込みのGPU全体とは範囲が異なる。
指令が未着のとき	位置・姿勢の差分をゼロとし、グリッパ指令のみ維持。相対移動指令を繰り返して移動を続けない。

低位制御は既存実装を再利用

LIBERO / robosuiteのOSCとグリッパ制御を用い、MuJoCoの接触物理へ反映した。独自IKの追加ではなく、認識から既存制御器までの接続と、非同期時の時刻・資源管理を工夫した。

MAIN COMPARISON / 20 TASK EPISODES

同期・非同期実行の比較

同期（lockstep）は推論中にシミュレーション時間を止める。**非同期（realtime）**は推論中も実経過時間に合わせ、50msごとに制御と物理演算を進める。



概念図。帯の長さは実測時間を表さない。主比較では先読みせず、行動列を使い切ってから次を要求する。

共通条件と評価時間

両モデルとも生成50行動・flow反復10回。先頭10行動を実行し、同じ配置1〜5・乱数を使用した。同期は最大280制御step（14シミュレーション秒）、非同期は最大14実秒または280step。重み読込み・準備運転2回・初回行動列の生成は計時外とした。

モデル / 方式	成功	方策生成・出力	生成＋ROS	待機	実測制御頻度
SmolVLA / 同期	**5/5**	578ms	615ms	—	7.13Hz
π0.5 / 同期	**5/5**	335ms	372ms	—	7.81Hz
SmolVLA / 非同期	**3/5**	508ms	604ms	53.2%	19.92Hz
π0.5 / 非同期	**5/5**	332ms	412ms	44.5%	19.98Hz

時間は初回を除く各試行平均の5試行平均。方策生成・保存済み出力処理・CUDA同期までの実経過時間で、入力前処理を除く。生成＋ROSは入力前処理と通信を含み、親プロセスの待ちを除く。同期の推論待ちは停止中なので待機tickとして数えない。

非同期の個別結果	配置1	配置2	配置3	配置4	配置5
SmolVLA / 非同期	成功	成功	未達	未達	成功
π0.5 / 非同期	成功	成功	成功	成功	成功

観測から実行までの遅れ

通常指令の観測取得→実行は、SmolVLA約938ms、π0.5約812ms。π0.5も待機44.5%を含み、20Hzの制御維持と新指令の連続供給は別の指標である。

比較の範囲

同期は両者5/5、成功数の差は非同期で確認された。方式間は撮像の更新位相も異なるため、差を推論遅延だけの効果と断定しない。各5配置であり、一般的な性能順位とは扱わない。

> **計画時の速度予想と実測の違い**
>
> この課題・実装では、π0.5は非同期の成功数が多く、方策生成・出力処理も短かった。モデル規模だけでは説明できないため、同じ保存観測を用いた追加計測で処理区間を分けた。

ISOLATED INFERENCE / SAME SAVED OBSERVATIONS

推論時間の内訳と速度差

同じ5観測を各5回使用し、撮像・ROS・制御を終了して1モデルずつ測定した。両者とも50行動・flow反復10回を維持し、各計測前に10回の準備運転を行った。

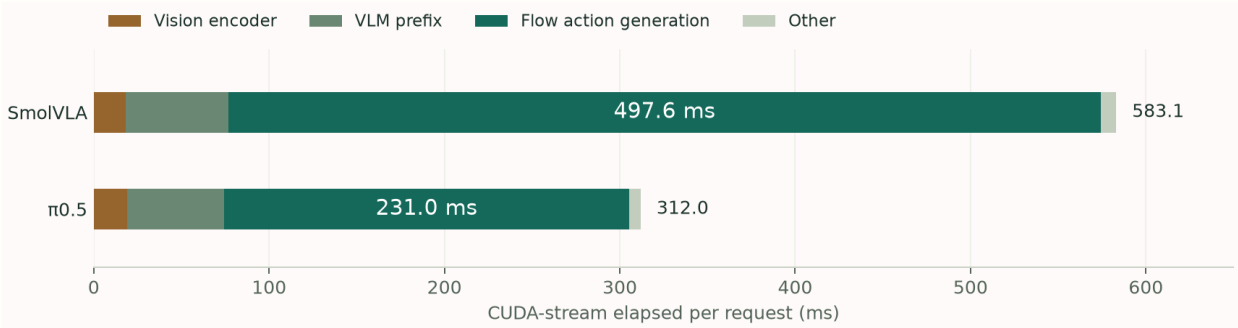
A　区間計測なし：入力前処理を含む呼出し全体

SmolVLA
603.6 ms

π0.5
367.9 ms

区間計測の前後で各25要求、計50要求の平均。π0.5は約39%短かった。前→後の対照平均はSmolVLA 585.1→622.1ms、π0.5 360.3→375.5msで、時間変動も記録した。

B　区間計測あり：CUDA stream上の経過時間



処理区間 / 各25要求平均	SmolVLA	π0.5
画像エンコーダ	18.2ms	18.9ms
観測理解：VLM prefix	58.6ms	55.4ms
行動生成：flow全体	497.6ms	231.0ms

各25要求の平均で、flowは10反復の合計。CUDA stream経過時間は実行指示の間隔や待ちも含み、GPU演算時間だけではない。区間計測ありの呼出し全体は637.7 / 341.7msだった。

画像処理とflow生成の比較

内部画像はSmolVLA 512×512、π0.5 224×224だが、画像エンコーダは約18〜19msだった。一方、flow生成は約498 / 231msであり、大きな差はこの区間に確認できた。行動expertは32層 / 18層で、幅やattention方式も異なる。繰り返す部分の構造は速度を考える手掛かりになるが、個別の寄与を分ける対照実験は未実施である。

精度と測定範囲を確認して解釈

代表的な行動expertのq_proj出力は両者BF16だった。SmolVLAを一律FP32とする説明は採用しない。区間計測の有無で両者の出力は完全一致した。前ページの508 / 332msは入力前処理を除く。計測範囲に加え並行負荷・準備運転も異なるため、この表を前ページの内訳としては扱わない。

CONTINUITY / RTC AND ADDITIONAL METHODS

行動列の接続を改善する検証

非同期生成では、新しい行動列が実行中の列と異なる動作を選ぶ場合がある。RTC（Real-Time Chunking）は、既存の予定指令と整合するよう生成中の行動変数を補正し、列の切り替えを滑らかにするための手法である。

既存の実行履歴・予定指令 → 遅れに合わせて生成中の列を補正 → 経過した指令を除いて切替

接続手法は別条件で比較した。生成50行動を保持し、残り30行動で次の推論を開始する（H50/R30）。同じモデル内で初期センサと最初の出力を照合し、補正なし・LeRobot公開RTC・生成中の行動変数への微分を追加した版を各5配置で評価した。重みは固定し、学習は行わない。

同じ基盤モデル内の条件	成功	生成＋ROS	待機割合
SmolVLA / 接続補正なし	**1/5**	615ms	0.25%
SmolVLA / LeRobot公開RTC	**1/5**	620ms	0.00%
SmolVLA / 入力微分を追加したRTC	**1/5**	1356ms	22.35%
π0.5 / 接続補正なし	**5/5**	367ms	0.00%
π0.5 / LeRobot公開RTC	**4/5**	455ms	0.00%
π0.5 / 入力微分を追加したRTC	**4/5**	890ms	0.00%

GPU使用を調整した再評価系列、計30試行。時間は初回を除く各試行平均の平均、待機は全tick合算。主比較の10行動・先読みなしとは待ち行列の条件が異なる。

> **切替時の指令差と、タスク成功を分けて評価**
>
> SmolVLAの公開RTCでは、同じ時点の旧予定と新採用の位置指令差が平均0.951→0.00869へ減少した。ただし成功は1/5のままだった。入力微分追加版は計算時間が増え、SmolVLAでは待機も22.35%へ増加した。π0.5は補正なし5/5に対し、RTCは両版4/5であり、成功数の改善は確認できなかった。

指令差はOSCの正規化位置指令のL2差（57 / 58切替）。メートル単位の位置誤差や、隣接する制御周期間の動作変化ではない。

異なる専用重みを使った追加検証

手法・基盤	比較した条件	公式成功
VLASH / 専用π0.5	画像と同時点 / 現在の自己状態 / 予測した自己状態	3/5・1/5・2/5
A2C2 / 作者450M SmolVLA	headなし / 残差として加算 / 直接行動として復号	4/5・1/5・0/5

通常のπ0.5・約605M SmolVLAとは基盤が異なり、主比較と合算しない。VLASHは状態時刻と行動の原点を対応づけた。A2C2は公開headと学習コードの厳密な対応が未確定であり、復号解釈の検証として扱う。

ENGINEERING CONTRIBUTION / LESSONS & REPRODUCIBILITY

工夫した点・得られた知見と応用可能性

技術課題	実装・切り分け	得られた知見
描画で周期を超過	シミュレータ制御とカメラを別プロセスに分離。取得時刻つき状態から撮像。	描画を待たず約20Hzで更新できた。撮像完了時刻を取得時刻へ代用しない。
モデルの速度予想	総時間に加え、同一観測で視覚・VLM prefix・flowを分けて計測。	小型モデルが常に速いとは限らず、今回の差は行動生成区間に集中した。
行動列の不連続	先読みの条件を固定した対照を置き、RTCの指令差と成否を別に評価。	指令の滑らかさ・待機短縮だけでは、把持・移載の成功は増えなかった。
通信のまれな待ち	A2C2追加検証では最終指令のROS往復を別プロセス化。期限と指令IDを検査。	通常の通信速度と遅い応答を区別。未着時は受信済み基盤指令へ戻した。

実用への接続

容器・部品の卓上移載が応用候補となる。既存の学習済み方策と制御器を用い、周期の異なる撮像・推論・制御を分離する設計は、限られたGPUでの試作に再利用できる。

評価の限界と次の検証

1課題・固定5配置のため、実機・未知物体・別タスクへの汎化は未検証である。撮像位相やOSの実行揺らぎも残る。今後は接触場面の切替誤差と、指令の観測時刻をそろえて検証する。

再利用した資産と検証可能性

資産	使用した役割・固定版	利用条件の記録
SmolVLA	SmolVLMを視覚言語基盤とする方策。LIBERO調整済み重み 6721902…	カード：Apache-2.0
π0.5	LIBERO調整済み公開重み 7152f13…、50行動・flow10反復	選択重み：Gemma条件
LeRobot / LIBERO	LeRobot 0.6.2、hf-libero 0.1.4、robosuite 1.4.0、MuJoCo 3.3.7	コード・重み・形状データの条件を個別記録
実行・通信	ROS 2 Jazzy / CycloneDDS、PyTorch 2.7.1+cu128、Ubuntu 24.04 / WSL2	依存版と取得元を固定

主比較20試行のROS受信内容364件を照合し、学習済み重みの復元と実行前後の不変を確認した。主比較、RTC、異なる専用重みの追加検証、推論内訳を別に保存し、条件と失敗結果を追えるようにした。

コードと測定記録

https://github.com/kenb91311/pai-final-44055-libero-vla

主比較：artifacts/realtime-vla/submission-smol-pi05/evidence-primary.json
RTC等・推論内訳：同フォルダのevidence-continuity.json。実行手順・取得条件はREADMEとdocs/に収録。

一次資料　SmolVLA重み ／ π0.5重み ／ LeRobot・RTC ／ LIBERO
モデル重み・tokenizer・外部形状データは配布せず、固定revisionと取得手順を記録した。